



MENTEES



Are you a biology major, pre-med or pre-health student, in an undergraduate or Master's program, interested in pursuing an advanced degree in biology or a related science, or a STEM student seeking guidance from someone with experience?

Here is how to become a BUMMP mentee for the 2026-27 academic year:

 Follow BUMMP on Instagram to see program & event updates

 Scan the QR codes below to take the next steps in joining the BUMMP family

 Get matched with a mentor based on shared interests, get to know them and ask all your questions (academics, research, etc), and gain a sense of belonging in the STEM community!



Visit our website to learn more!



Fill out this mentee interest form to get on our email list!



Click the link in our bio for more resources!!



STARTneuro information session

Are you a **community college transfer student** who is curious about **neuroscience**?

Learn about the STARTneuro program, which could provide you with mentorship and funding to pursue research.

October 8th @ 5-6 pm
2130 Bonner Hall

Fill out this interest form to keep in touch:
<https://forms.gle/hvsf13TV2Ac5cDSZ7>

More information:
startneuro.ucsd.edu



Funded by NIH BP-ENDURE PROGRAM

Where Python lives, and how to talk to it

BILD 62

Objectives for today

- Identify various ways of writing and running Python code
- Learn basics of Python syntax
- Define three types of variables: integers, floats, and strings
- Concatenate & slice strings
- Determine rules for variable names

There are multiple ways to interact with the Python interpreter

- Command line (terminal)
 - Line-by-line coding
 - Running “Scripts”

“Terminal” comes from the days *before* desktop computers, when a computer occupied a set of cabinets or even an entire room. A terminal was a device with a (text-only) monitor and keyboard whereby a user could control the computer from a distance over a dedicated, wired connection.



A DEC VT100 terminal at the Living Computer Museum (apparently connected to the museum's DEC PDP-11/70 mainframe computer). Source: [Wikipedia](#)

Linux



```
Terminal
File Edit View Search Terminal Help
user@linux-computer:~$ python MyPythonScript.py
user@linux-computer:~$ python MyPythonScript.py param1 param2
user@linux-computer:~$
```

Macintosh



```
Users — bash — 86x14
User-Imac:python MyPythonScript.py
User-Imac:python MyPythonScript.py param1 param2
User-Imac:~
```

Windows



```
Command Prompt
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
C:\Users\UserName>python PythonScript.py param1 param2
```



Right-click
Edit with IDLE

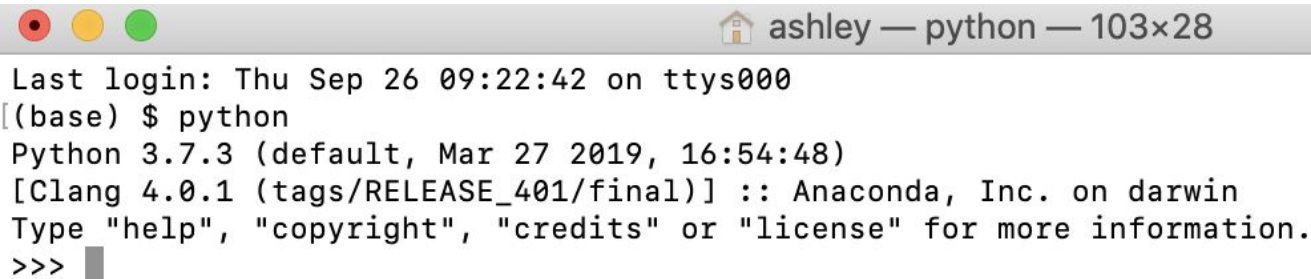
```
PythonScript.py - C:\Users\Tim Stevens\Desktop\PythonScript.py
File Edit Format Run Options Windows Help
def calcSeqIdentity(seqA, seqB):
    numPlaces = min(len(seqA), len(seqB))
    score = 0.0
    for i in range(numPlaces):
        if seqA[i] == seqB[i]:
            score += 1.0
    return 100.0 * score/numPlaces
Ln: 1 Col: 0
```

Running a Python script from
different operating systems

(from <http://www.cambridge.org/pythonforbiology>)

If you have a Mac

- Macs ship with Python already installed.
- You can check which version by opening **Terminal** & typing **python --version**

A screenshot of a macOS Terminal window. The title bar shows three colored window control buttons (red, yellow, green) on the left, and a home icon followed by the text 'ashley — python — 103x28' on the right. The terminal content shows the command 'python' being executed, which displays the Python version (3.7.3) and other system information. The prompt '>>>' is visible at the bottom of the terminal output.

```
Last login: Thu Sep 26 09:22:42 on ttys000
(base) $ python
Python 3.7.3 (default, Mar 27 2019, 16:54:48)
[Clang 4.0.1 (tags/RELEASE_401/final)] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

The “>>>” tells you you’re inside the Python prompt, and the computer is ready for some code!

There are multiple ways to interact with the Python interpreter

- Command line
 - Line-by-line coding
 - Running “Scripts”
- Integrated Development Environments
 - Each have pros/cons, and people just have their own preferences.
 - A few good options are:
 - Visual Code (<https://code.visualstudio.com/download>)
 - Spyder (Included with Anaconda)
- Jupyter Notebook — *most of what we'll do in this course*

Integrated Development Environments (IDEs)

- Help you write, debug, and compile code
 - **Compiling** is the process of translating your **source code** into **machine code**
- More features, such as auto-completion, memory for commands, a variable/function explorer, and often version control

A few notes

Macs have a native installation of Python.

- It may be older & will not include the extra packages that you will need for this class, and is best left untouched.
- Downloading via Anaconda or VS Code may install a separate, independent install of Python, leaving your native install untouched.

Windows does not require Python natively and so it is not typically pre-installed.

If you're not sure which Python your computer is using, ask it (in Python):

```
>>> which python
```

Objectives for today

- Identify various ways of writing and running Python code
- **Learn basics of Python syntax**
- Define three types of variables: integers, floats, and strings
- Concatenate & slice strings
- Determine rules for variable names

There are different types of programming languages, each with their own syntax, or rules.

- **Syntax:** the rules of a programming language
 - Includes punctuation, spacing, indentation, etc.
- Each language has strengths & weaknesses.
- Regardless, each language ultimately needs to communicate with the hardware of the computer, in 1's and 0's.
 - It's similar to DNA! And similar to DNA, we don't often describe it in individual base pairs. Instead we describe genes and describe DNA in a higher level way.

Storing values

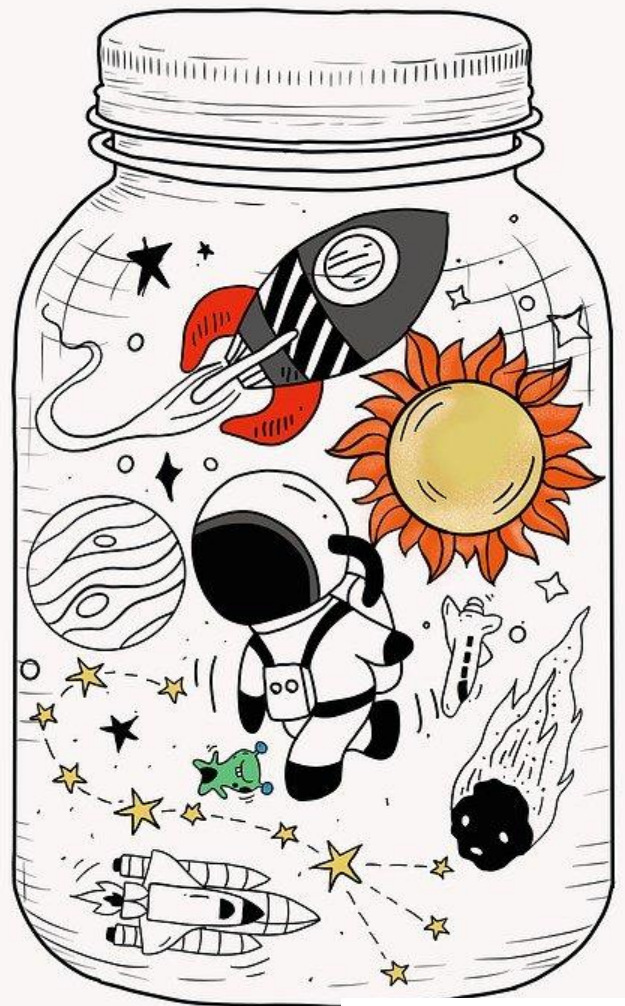
We can store values in variables, e.g.:

```
variable_1 = 48
```

← name ← value

Variables can be text, integers, or floats (with decimals), e.g.:

```
text_string = 'hello'
```



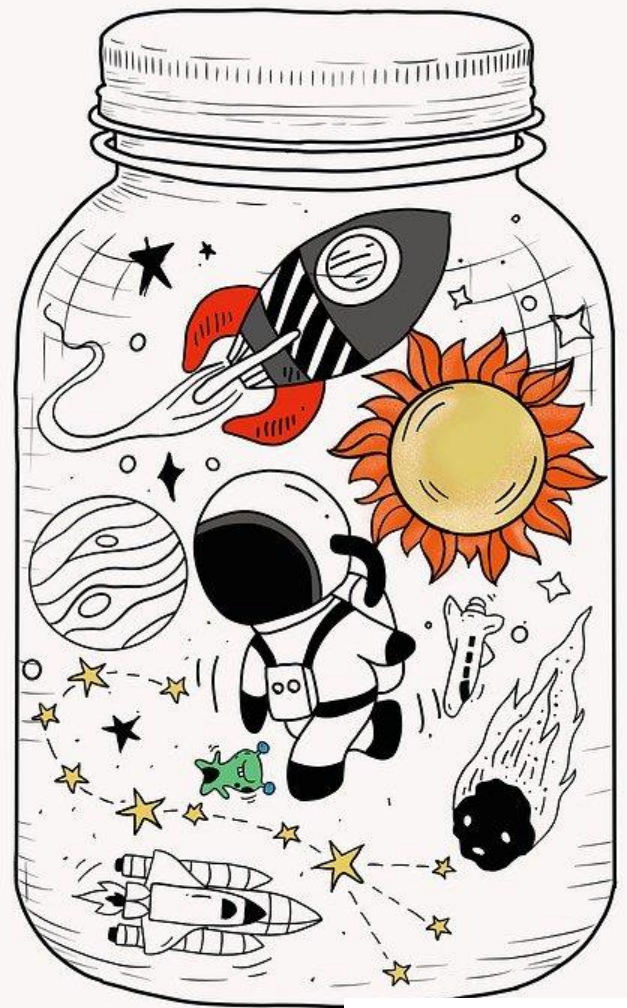
Storing values

We can store values in variables, e.g.:

```
variable_1 = 48
```

We use an equal sign to *assign* the value to a name, but it's not the same thing as saying they are equal.

In other words, we're storing that value in the variable. (Think of them like cookie jars)



Creating new variables

- Names are always on the left of the `=`, values are always on the right
- Pick names that describe the data / value that they store
- Make variable names as **descriptive** and **concise** as possible (this is an art!)
- Variables cannot be Python keywords:

```
[>>> import keyword  
[>>> print(keyword.kwlist)  
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def',  
 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda',  
 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']  
>>> █
```

(There are other rules for variable names....)

Python has many variable types, and each function a little bit differently.

Understanding your variable type is crucial for working with it.



Reals



Explanation of variable types

Built-in simple variable types in Python

Type	Example	Description
<code>int</code>	<code>x = 1</code>	integers (i.e., whole numbers)
<code>float</code>	<code>x = 1.0</code>	floating-point numbers (i.e., real numbers)
<code>complex</code>	<code>x = 1 + 2j</code>	Complex numbers (i.e., numbers with real and imaginary part)
<code>bool</code>	<code>x = True</code>	Boolean: True/False values
<code>str</code>	<code>x = 'abc'</code>	String: characters or text
<code>NoneType</code>	<code>x = None</code>	Special object indicating nulls

Integers, strings, floats (summary)

function to convert to integer

- **Integers** (**int**): any whole number
- **Float** (**float**): any number with a decimal point (floating point number)
- **String** (**str**): can contain letters, numbers, symbols, spaces
 - Represented by matching beginning & ending quotes
 - Quotes can be single or double; use single *within* double
 - Use \ to ignore single quote
 - Concatenate strings with +

Checking variable types

This is a very useful troubleshooting step!

- You can check what type your variable (**a**) is by using **type(a)**
 - Alternatively, we can use:

```
>>> type(a) is float
```

or

```
>>> isinstance(x, float)
```
- Python lets you change the type of variables, however, ***you cannot combine types.***
- Use **del** to delete variables

It's important to know the precision of your variables.

In most datasets, we are working with floats.



Autopsy Report:

Dr. Andrew Esty

Time of Death: 03/16 11:53

Cause of Death: Rounding Errors

Historical sidenote

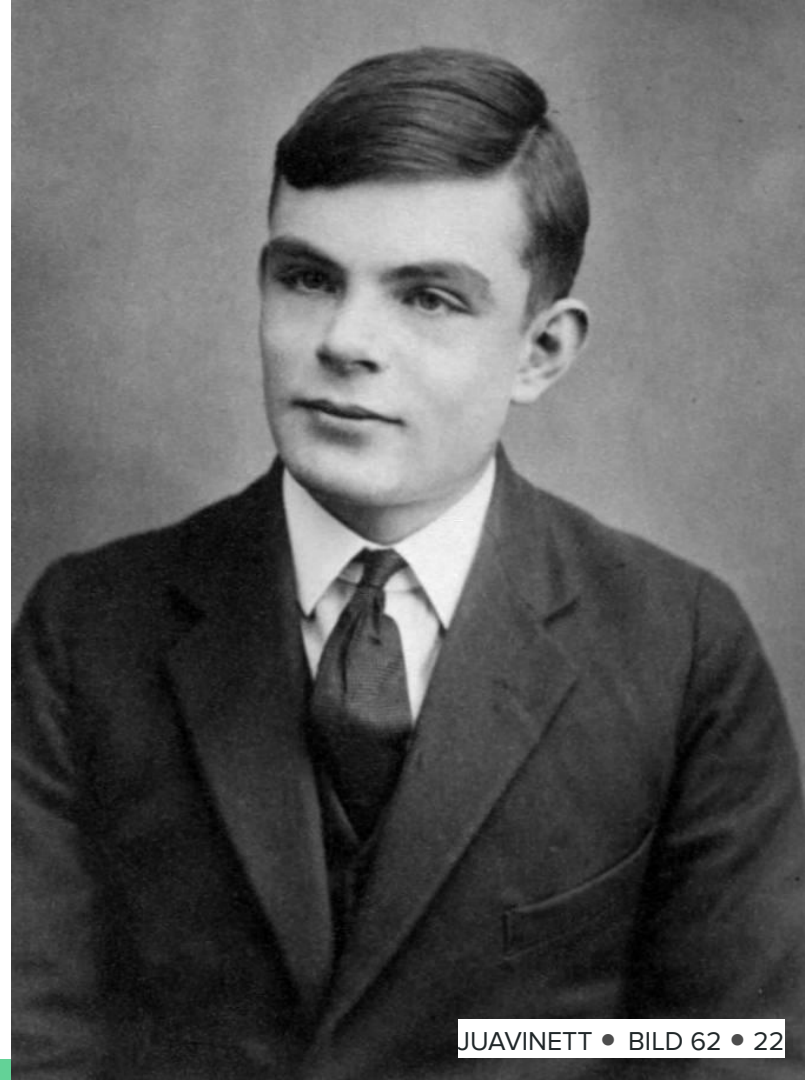
- A **cipher** is a procedure for **encoding** and **decoding** messages or data.
- By manipulating strings, it's quite straightforward to create an encoder using a few lines of code.
- During WWII, Germany used a **variable encoder** which changes its encoding strategy each time it runs.



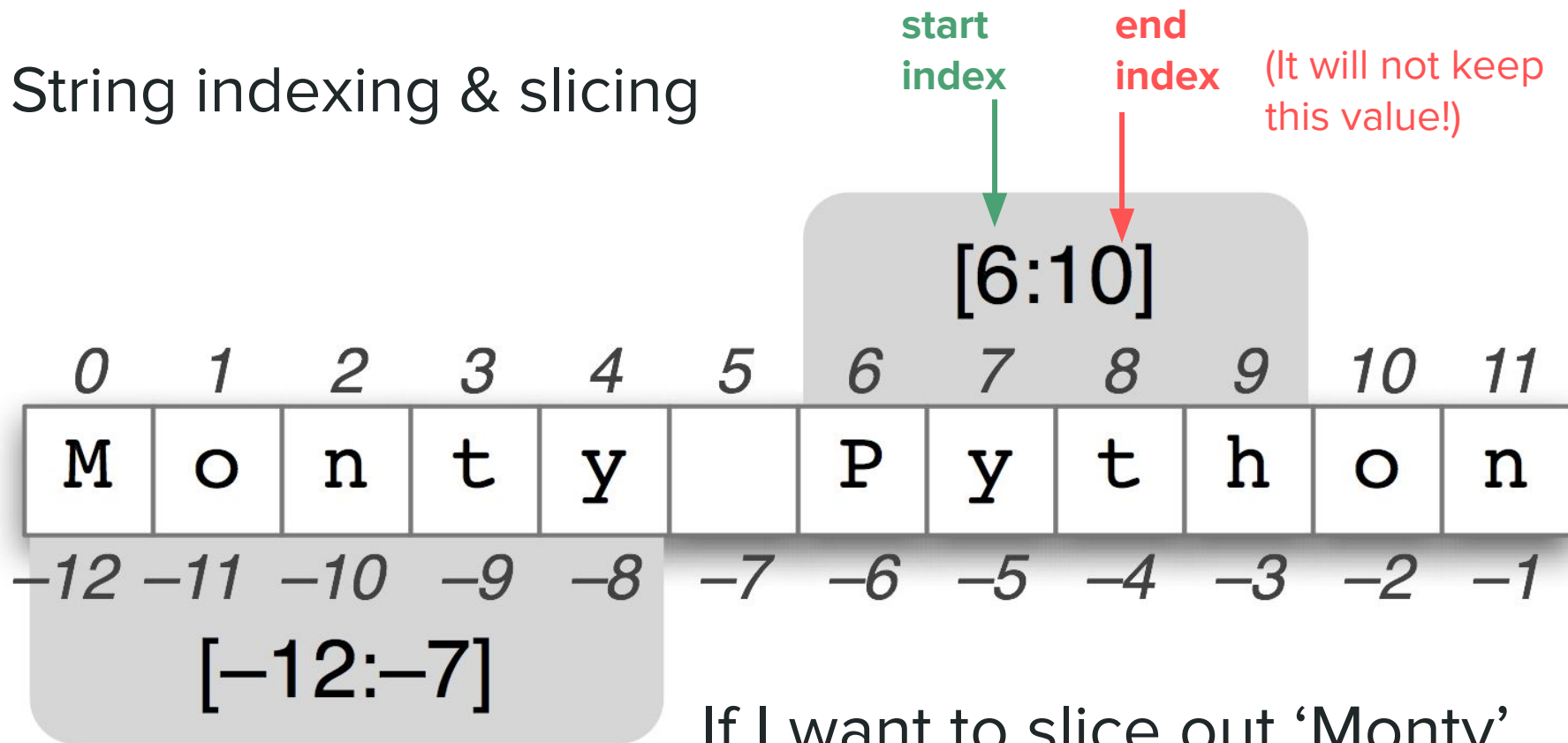
Historical sidenote

- A team led by **Alan Turing** built & programmed machines that could crack the ENIGMA code, ultimately shortening the war & saving many lives.
- Alan Turing went on to make many contributions to computer science until he was prosecuted by the British Government for “homosexual activity”

Note: The Imitation Game is an Oscar-worthy depiction of his life and work (but takes some dramatic liberties...)



String indexing & slicing



If I want to slice out 'Monty',
I would use `[0:4]` or `[:4]`

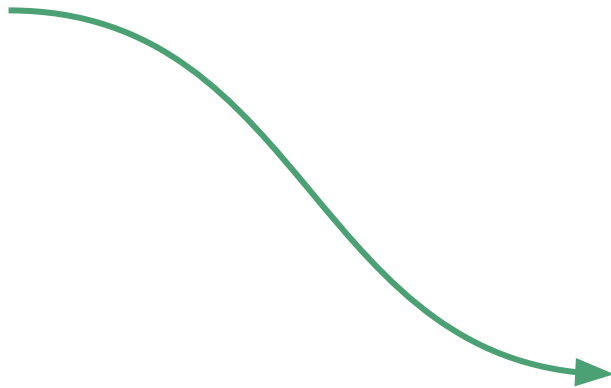
Why does Python count from 0?

Programming languages like Fortran, MATLAB and R start counting at 1 because that's what human beings have done for thousands of years!

Languages in the C family (including C++, Java, Perl, and Python) count from 0 because it represents an offset from the first value in the array (the second value is offset by one index from the first value).

This is closer to the way that computers represent arrays.

Conceptual check in



Let's get into a Jupyter notebook!
Use the magic link to sync up your DataHub
with our folder, and open notebook 02.

You'll also need to open the quiz on Canvas.

Resources

Jupyter Notebooks:

- [Official Jupyter documentation](#)
- [Example notebooks](#)
- [A Gallery of Interesting Jupyter Notebooks](#)
- [Software Carpentry: Running & Quitting Jupyter Notebooks](#)
- [A List of Good Python YouTube Channels](#)
- [Python Fundamentals](#) (Software Carpentries)
- [Error types in Python](#)